



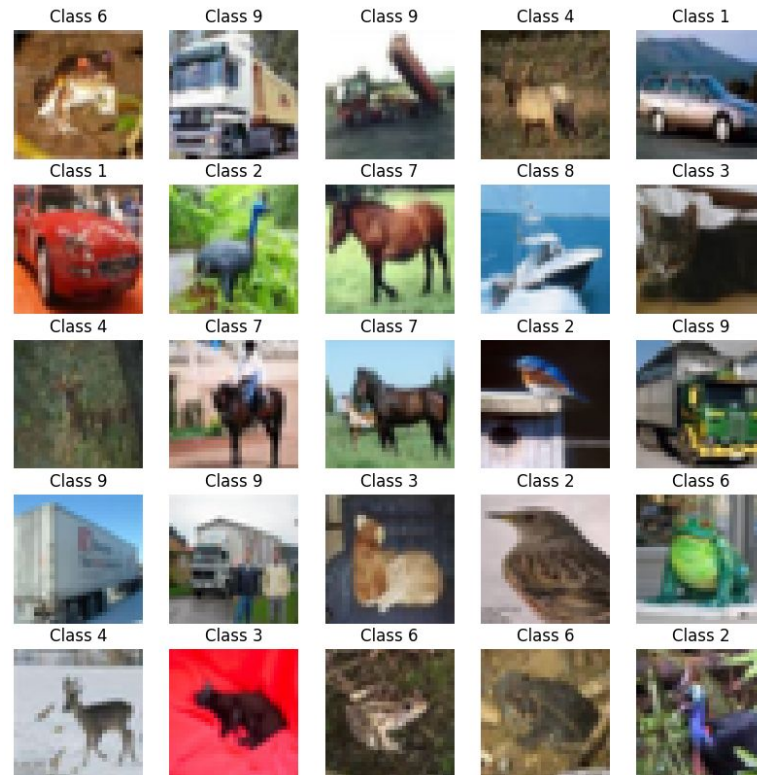
Application of Convolution Neural Networks for image classification: CIFAR10

Vítor Ferraz
Satishbabu Rajagopal
Nicole Segura

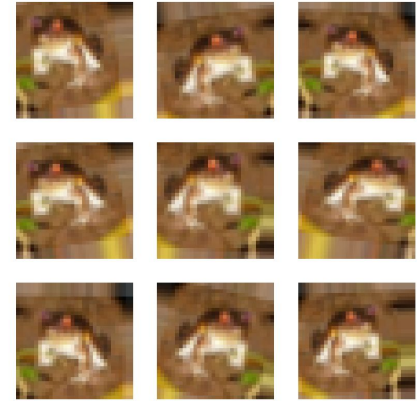
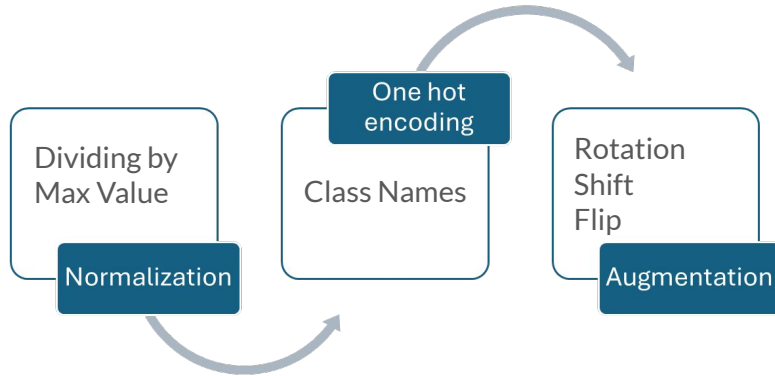
Custom CNN: Input Dataset

CIFAR-10 input data

50,000 training images and 10,000 test images
32×32 colour images with 3 channels
10 target classes



Custom CNN: Data Preparation and Preprocessing



Key insight

The model was trained on small, low-resolution images, so preprocessing and augmentation were important to help the CNN generalise better.

Rotation: 15°
Width and height shift: 0.1
Zoom: 0.1
Horizontal flip



Initial Custom CNN Architecture

Conv 2D(3x3)

MaxPooling(2,2)

Conv2D(3x3)

MaxPooling(2,2)

Flatten()

ReLu

ReLu

Custom CNN Architecture Improvement



Experimentation Strategy

- Started with a simple baseline CNN
- Evaluated learning curves after each experiment
- Looked for:
 - training vs validation accuracy gap
 - validation loss behaviour
 - signs of underfitting or overfitting
- Changed one major component at a time where possible

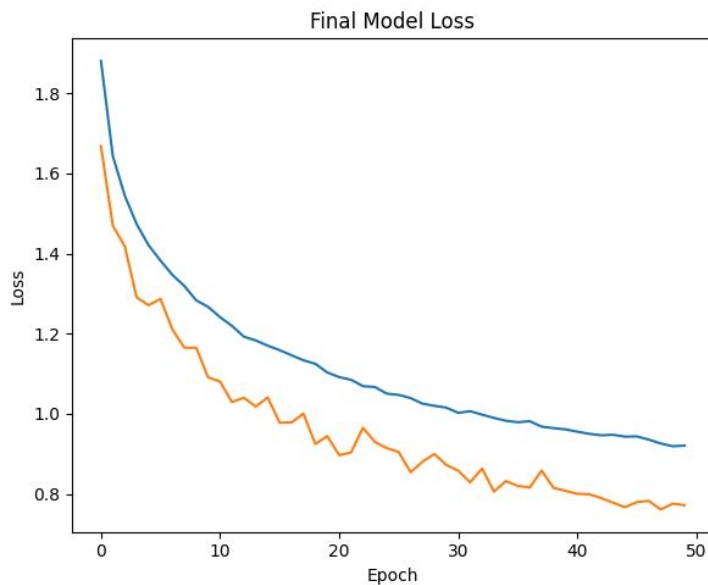
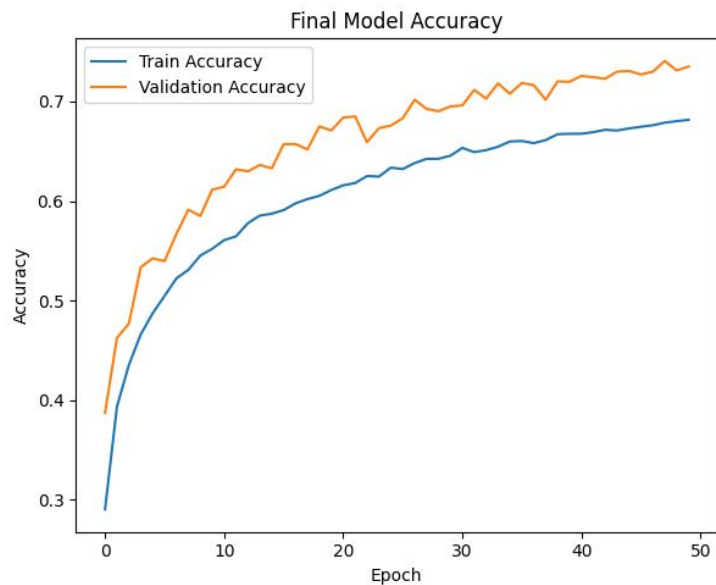
Main Improvement Levers

- Learning rate tuning
- Increasing filters
- Adding convolutional layers
- GlobalAveragePooling2D
- Dropout regularisation
- Data augmentation
- EarlyStopping and ModelCheckpoint
- ReduceLROnPlateau for adaptive learning rate reduction

Key insight

The learning curves guided the next experiment. We did not tune randomly; we used training behaviour to decide what to change.

Best Custom CNN: Final Model and Performance



Best Custom CNN: Some Confusion Pairs



Confusions: Cat ↔ Dog



Confusions: Bird ↔ Deer



Confusions: Horse ↔ Deer



Best Custom CNN: Final Model and Performance



Final Best Custom CNN

Conv2D 32 → MaxPooling

Conv2D 64 → MaxPooling

Conv2D 128

GlobalAveragePooling2D

Dropout 0.4

Dense output layer with 10 classes

Trained with data augmentation

EarlyStopping, ModelCheckpoint, and ReduceLROnPlateau used during training

Final Test Performance

Accuracy: ~0.73

Precision: ~0.74

Recall: ~0.73

F1-score: ~0.73

Class-Level Behaviour

- Stronger classes: automobile, ship, truck
- Weaker/confused classes: cat, dog, bird, deer
- Common challenge: visually similar classes in small 32×32 images

Key insight

The final custom CNN became a strong learning baseline, but it still had to learn all visual features from scratch.

Observation

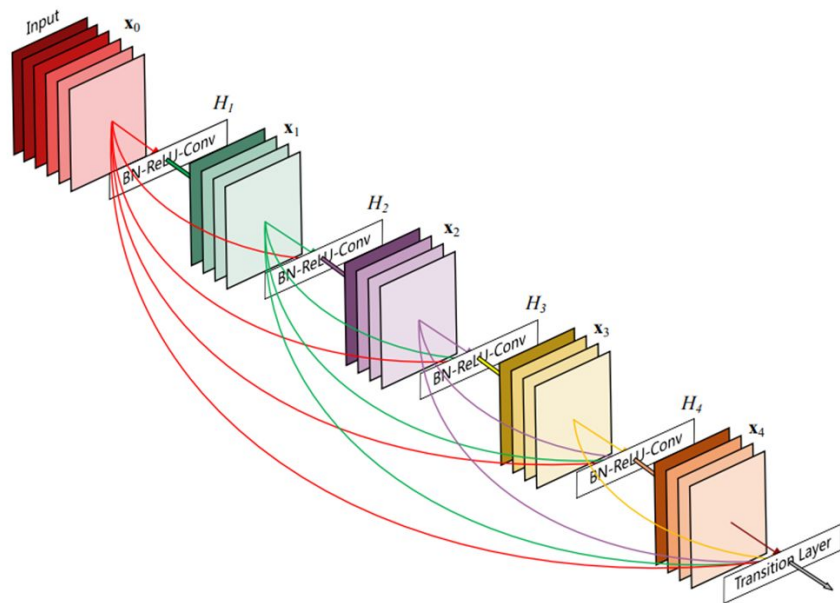
This is why it became the right benchmark for the next question: can **transfer learning** outperform our best custom CNN?



Transfer Learning

MobileNetV3, DenseNet121, EfficientNetB0

Dense Network Connectivity (DenseNet121)



Mechanism:

In a feed-forward CNN:

$$x_\ell = H_\ell(x_{\ell-1})$$

In a Residual CNN:

$$x_\ell = H_\ell(x_{\ell-1}) + x_{\ell-1}$$

In a Dense CNN:

$$x_\ell = H_\ell([x_0 + x_1 + \dots + x_{\ell-1}])$$

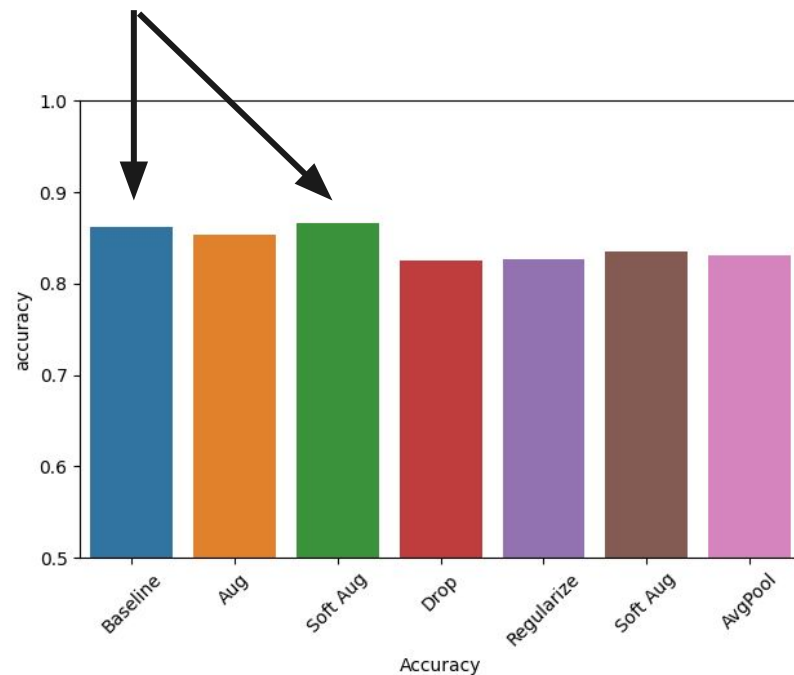
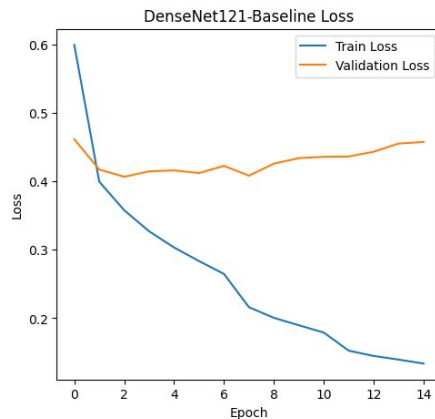
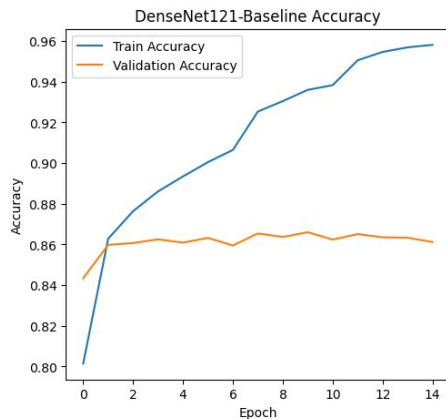


Transfer learning with DenseNet on CIFAR10

1. Minimum preprocessing data:
 - a. One hot encoding labels
 - b. Resizing to 96x96
2. To avoid high memory demands, further preprocessing was only called during model training:
 - a. Adding as layers: Resizing, Lambda for preprocessing_input with DenseNet specifications as input for the DenseModel.

Challenges in workflow

- Baseline: Model heavily overfits



Step 1. Data Augmentation in CIFAR10

- Main challenge: Memory crash
- Small images with strong data augmentation → underfitting
- Calling augmentation as → produces each time a different image

Original



Augmented



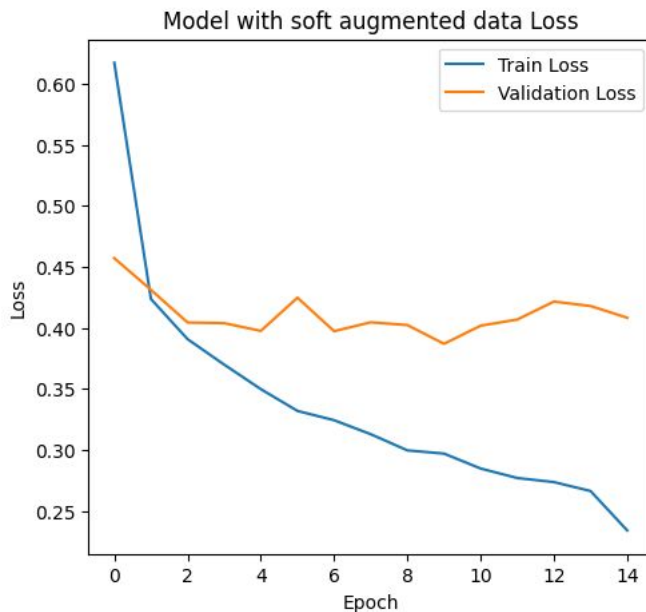
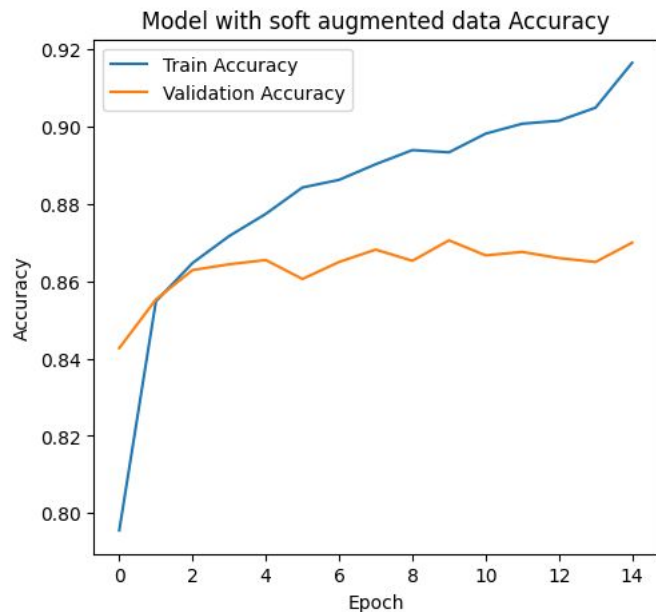
Original



Soft Augmented

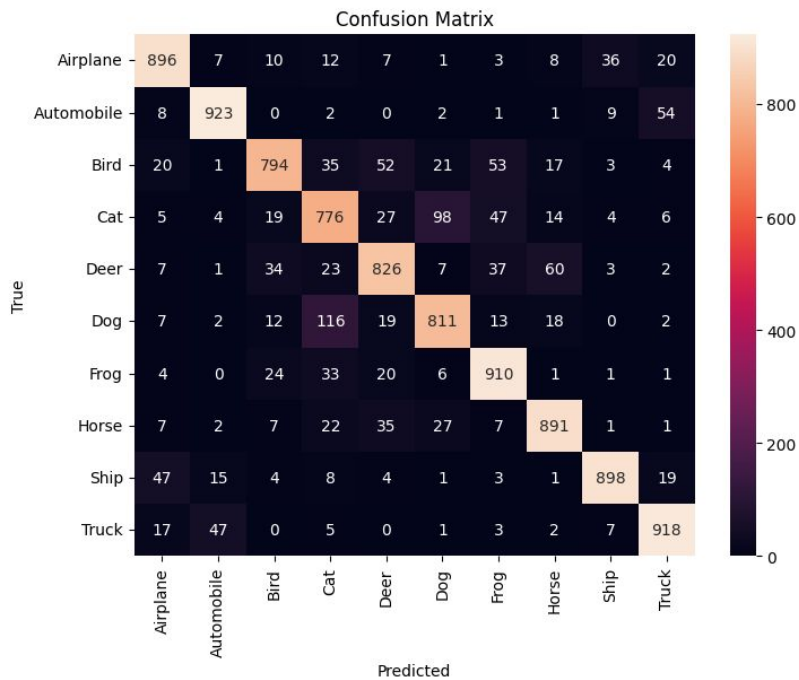


Soft Augmentation



Evaluation on test data
Accuracy: 0.86
Loss: 0.44

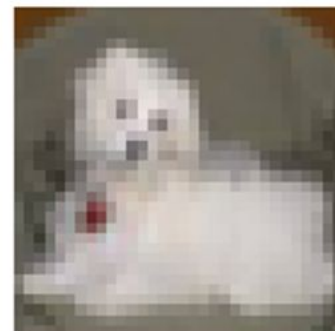
Soft Augmentation



True: Cat
Pred: Dog



True: Dog
Pred: Cat



True: Deer
Pred: Horse

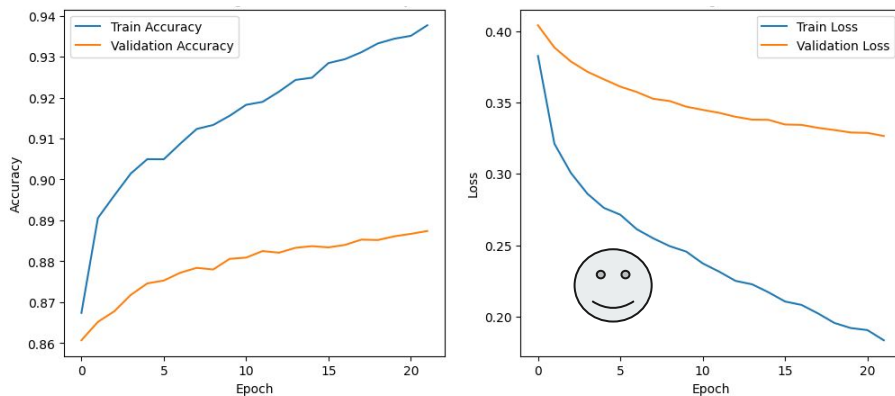


True: Horse
Pred: Deer



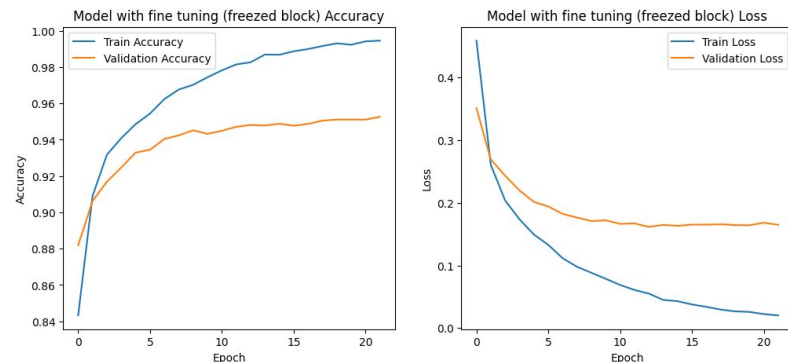
Fine tuning on DenseNet121

- Freezing only the last 40 layers



Evaluation on test data
Accuracy: 0.88 , Loss: 0.34

- Freezing only the last dense block



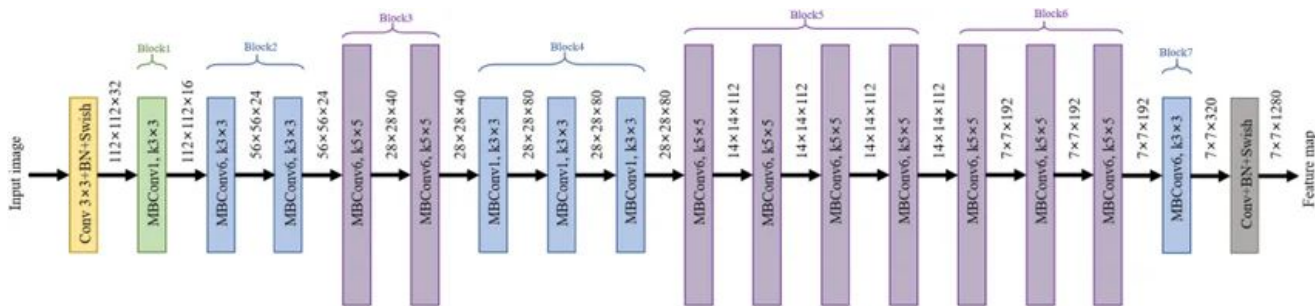
Evaluation on test data
Accuracy: 0.95 , Loss: 0.16

Best approach: softer augmentation → correct fine-tuning → lower LR → Better freezing strategy

EfficientNetBo Architecture: Pretrained Feature Extractor

- EfficientNetBo is built for MBConv blocks.
- These blocks progressively reduce image resolution while increasing feature depth.
- In our project, we used the pretrained EfficientNetBo base as a feature extractor.
- We replaced the original classifier with a new CIFAR-10 classification head.

NOTE: Our goal was not to reproduce the model paper's benchmark accuracy, but to test whether pretrained EfficientNetBo features improved performance over our best custom CNN under our project constraints.



Illustrative (unofficial) EfficientNetBo architecture diagram showing MBConv block structure.

Why We Compared EfficientNetBo Against Our Best Custom CNN

PURPOSE OF THIS COMPARISON

- Our best custom CNN was trained from scratch on CIFAR-10.
- EfficientNetBo was used as a transfer learning model with ImageNet pretrained weights.
- Both models were evaluated on the same CIFAR-10 test set.
- The goal was to compare:
 - learning from scratch vs.
 - reusing pretrained visual knowledge.

KEY INSIGHT

EfficientNetBo is not just "a bigger CNN". It starts with visual patterns already learned from a much larger dataset.

Best Custom CNN	EfficientNetBo
Learns from CIFAR-10 only	Starts with ImageNet knowledge
Uses 32×32 images	Uses resized 96×96 images
Built by us	Pretrained architecture

OBSERVATION

This makes the comparison useful because it shows the practical difference between custom architecture design and transfer learning.

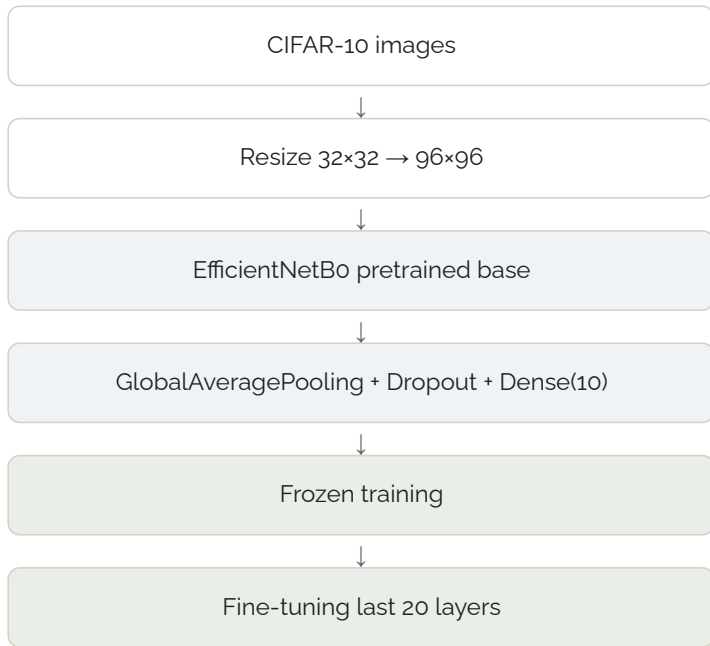
EfficientNetBo Workflow: Frozen Model → Fine-Tuning

EFFICIENTNETBo TRANSFER LEARNING APPROACH

- Loaded EfficientNetBo with ImageNet weights.
- Removed the original top classifier.
- Added a new CIFAR-10 classification head.
- First trained the model with the EfficientNetBo base frozen.
- Then fine-tuned only the last 20 layers.
- Used a very low learning rate: 1e-5.

KEY INSIGHT

The frozen model tested whether ImageNet features were already useful for CIFAR-10. Fine-tuning then allowed limited adaptation to our dataset.



OBSERVATION

Fine-tuning was done conservatively to avoid damaging the pretrained weights or catastrophic forgetting-induced behaviour.

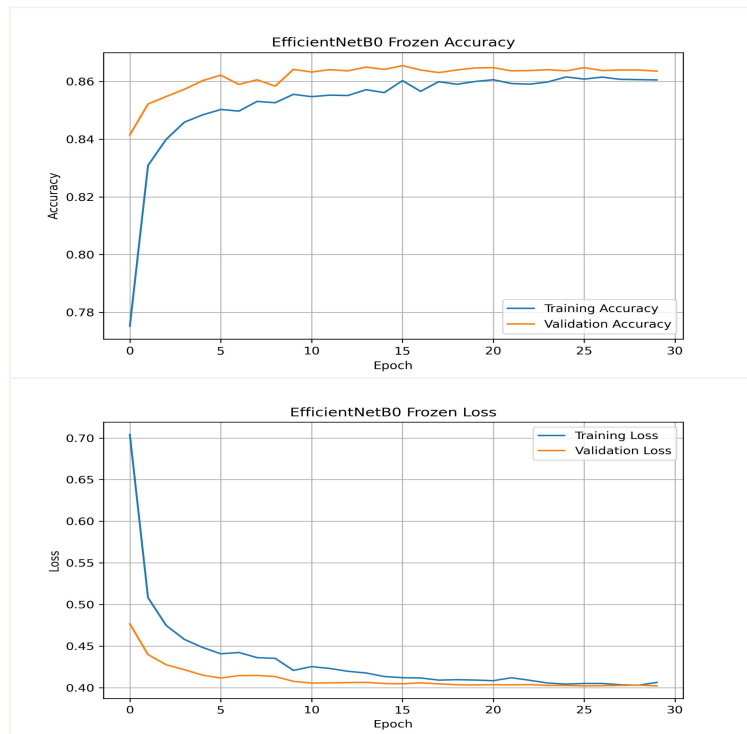
Learning Behaviour and Evaluation (Frozen)

WHAT WE OBSERVED FROM FROZEN EFFICIENTNETB0

- EfficientNetB0 was loaded with ImageNet pretrained weights.
- The base model was kept frozen.
- Only the new CIFAR-10 classification head was trained.
- CIFAR-10 images were resized from 32×32 to 96×96.
- This tested how useful pretrained visual features were before fine-tuning.

KEY INSIGHT

The frozen model already performed strongly, showing that ImageNet features transferred well to CIFAR-10.



OBSERVATION

The accuracy curve improved quickly in the first few epochs and then plateaued after around 10-15 epochs, stabilising in the mid-to-high 80% range. This suggested that the frozen model was already a strong feature extractor, but fine-tuning could still provide a small additional improvement.

Learning Behaviour and Evaluation

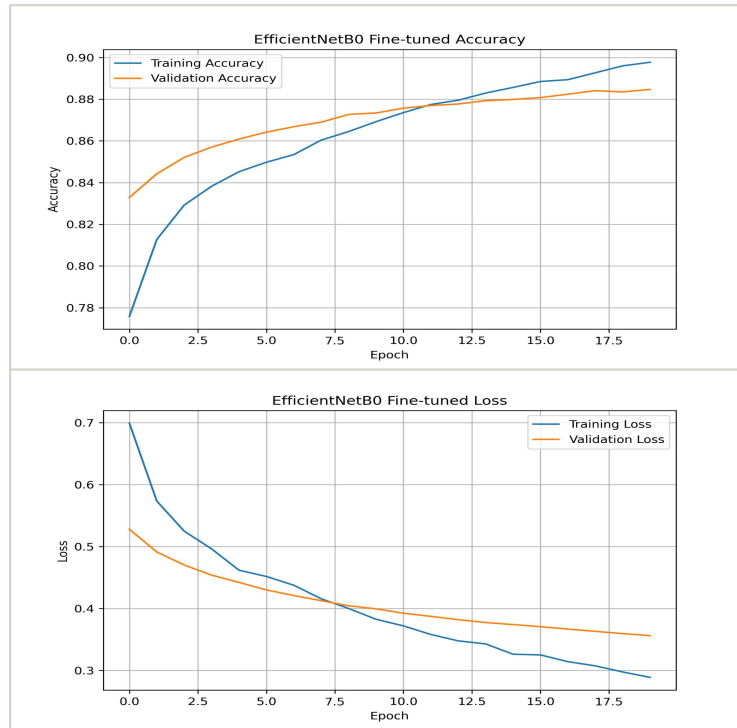


WHAT WE OBSERVED FROM EFFICIENTNETB0 FINE-TUNED

- Frozen EfficientNetB0 already performed strongly.
- Fine-tuning improved validation performance slightly.
- The fine-tuned model reached around 88% test accuracy.
- Validation and test performance were consistent.
- Mild overfitting appeared, but not enough to reject the model.

KEY INSIGHT

The fine-tuned model generalised well because validation and test results were aligned.



OBSERVATION

The model's main weakness was not overall accuracy, but class-level confusion between visually similar categories.

Best Custom CNN vs EfficientNetBo Fine-Tuned

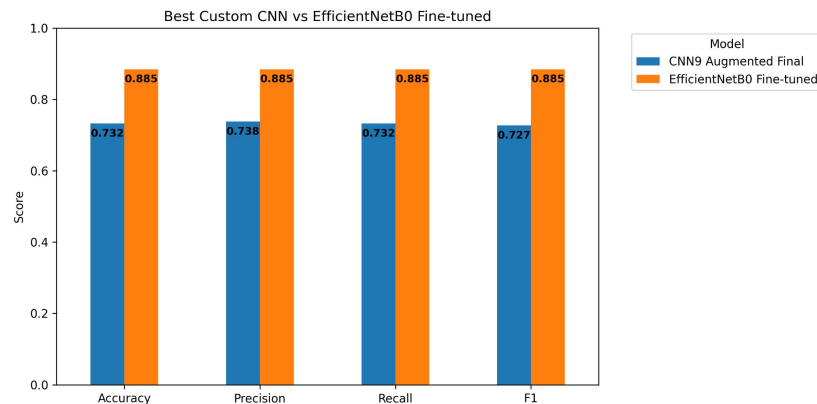
FINAL COMPARISON

Model	Accuracy	Precision	Recall	F1
Best Custom CNN	~ 0.73	~ 0.74	~ 0.73	~ 0.73
EfficientNetBo Fine-Tuned	~ 0.88	~ 0.88	~ 0.88	~ 0.88

- EfficientNetBo outperformed the best custom CNN across all metrics.
- The best custom CNN performed reasonably, but had to learn visual features from scratch.
- EfficientNetBo benefited from pretrained ImageNet features.

KEY INSIGHT

Transfer learning gave a clear performance advantage because the model started with reusable visual knowledge.

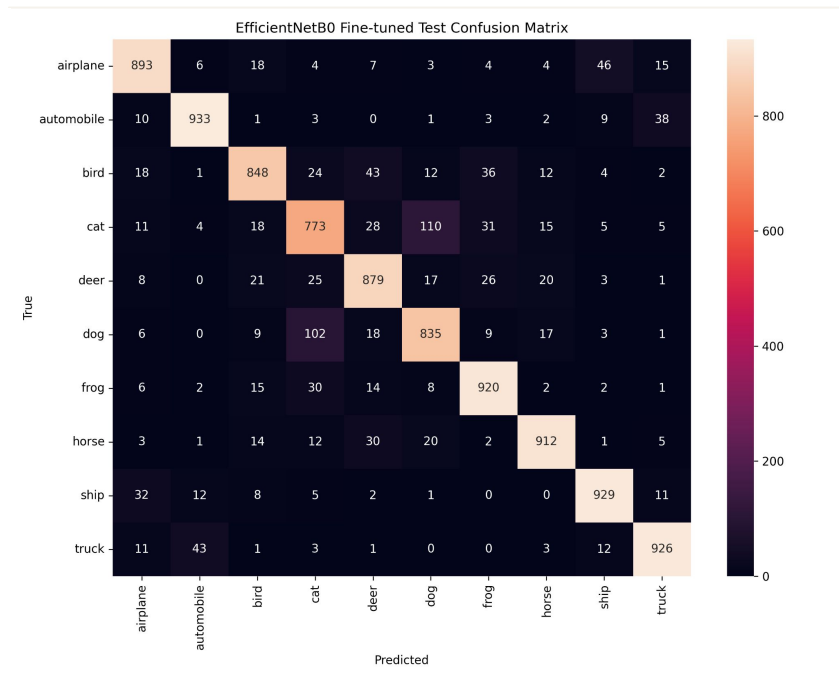
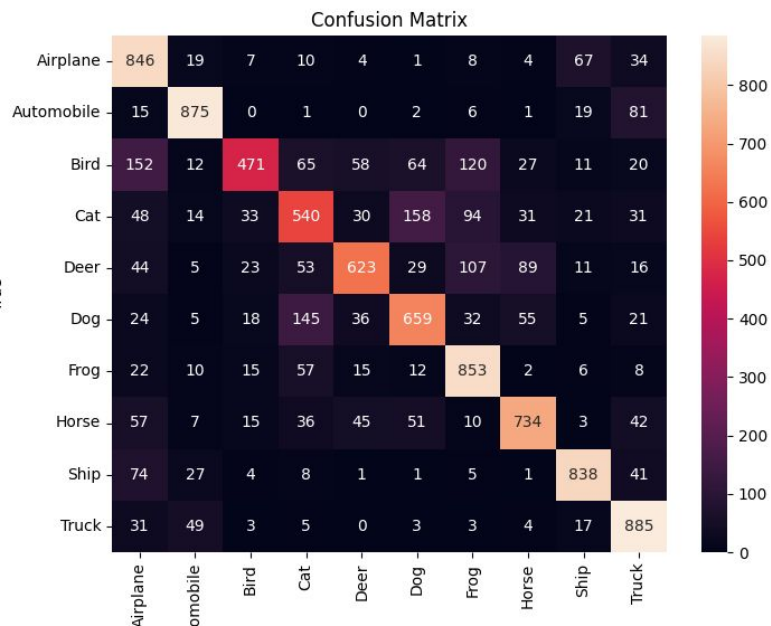


OBSERVATION

The custom CNN was more transparent and useful for learning architecture behaviour, but EfficientNetBo delivered stronger final performance.

Class-Level Performance: Custom CNN vs EfficientNetBo

Confusion matrices show where each model makes class-specific errors.



Class-Level Insights from EfficientNetB0 Fine-Tuned

WHERE EFFICIENTNETB0 PERFORMED WELL AND WHERE IT STRUGGLED

Strong classes

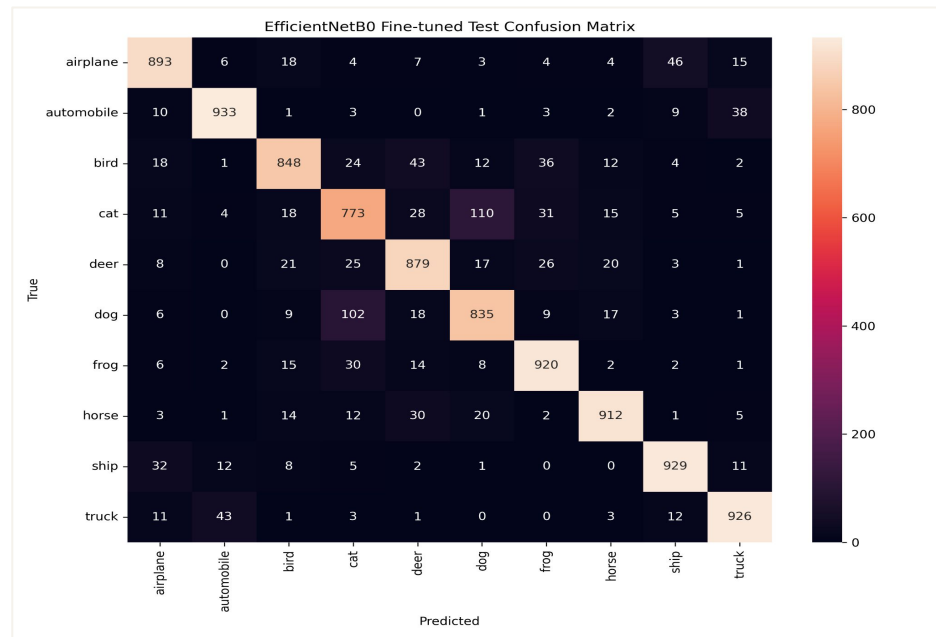
automobile
ship
truck
frog
horse

Weaker classes

cat
dog
bird

Main confusion

cat ↔ dog
automobile ↔ truck
airplane ↔ ship



KEY INSIGHT

The errors were not random. They were concentrated in visually similar classes.

OBSERVATION

This is expected with CIFAR-10 because the images are small, low-resolution, and sometimes visually ambiguous.

Understanding the Errors: Key Confusion Pairs - EfficientNetBo (Fine-Tuned)

Most errors occur between visually similar CIFAR-10 classes.

True: cat
Pred: dog



True: cat
Pred: dog



True: cat
Pred: dog



True: cat
Pred: dog



True: cat
Pred: dog



Potential EfficientNetBo Improvements



- Test larger input sizes: $96 \times 96 \rightarrow 128 \times 128$, if RAM allows.
- Combine resizing with data augmentation.
- Fine-tune more EfficientNetBo layers instead of only the last 20.
- Compare fixed low learning rates: $1e-5$ vs $1e-4$.
- Use ReduceLROnPlateau only as a support tool, not as the main learning-rate strategy.
- Evaluate whether less conservative fine-tuning improves CIFAR-10 performance.

Key takeaway

- Our EfficientNetBo setup was intentionally conservative.
- A stronger future experiment would test whether CIFAR-10 benefits from larger images, augmentation, and unfreezing more pretrained layers.

MobileNet & MobileNetV3 Transfer Learning – Overview, Insights & Trade-Offs



Implemented Transfer Learning using MobileNet and MobileNetV3Small
Compared against a Custom CNN model on the CIFAR-10 dataset
Applied preprocessing, augmentation, and hyperparameter optimization.

Evaluated using : Accuracy, Precision, Recall, F1-Score

Architecture Insights

MobileNet

Lightweight CNN using Depthwise Separable
Convolutions
Reduced memory and computational cost

MobileNetV3Small

Improved version of MobileNet
Better latency-to-accuracy tradeoff

Training Pipeline

Input → Augmentation → Resize →
Preprocessing
→ Pretrained Backbone
→ GlobalAveragePooling
→ Dense Classifier
→ Softmax Output

MobileNet Models Compared with Custom CNN

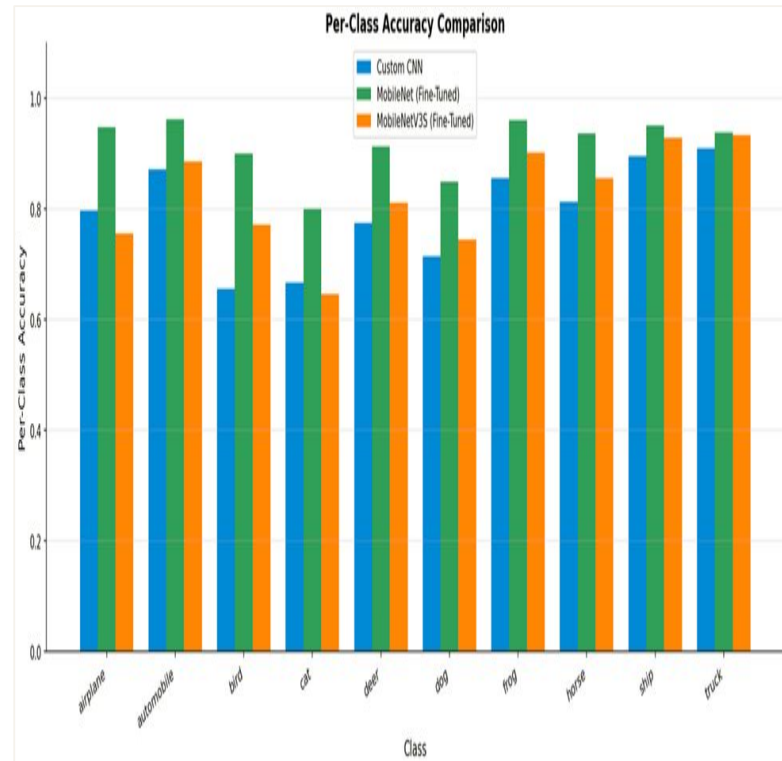
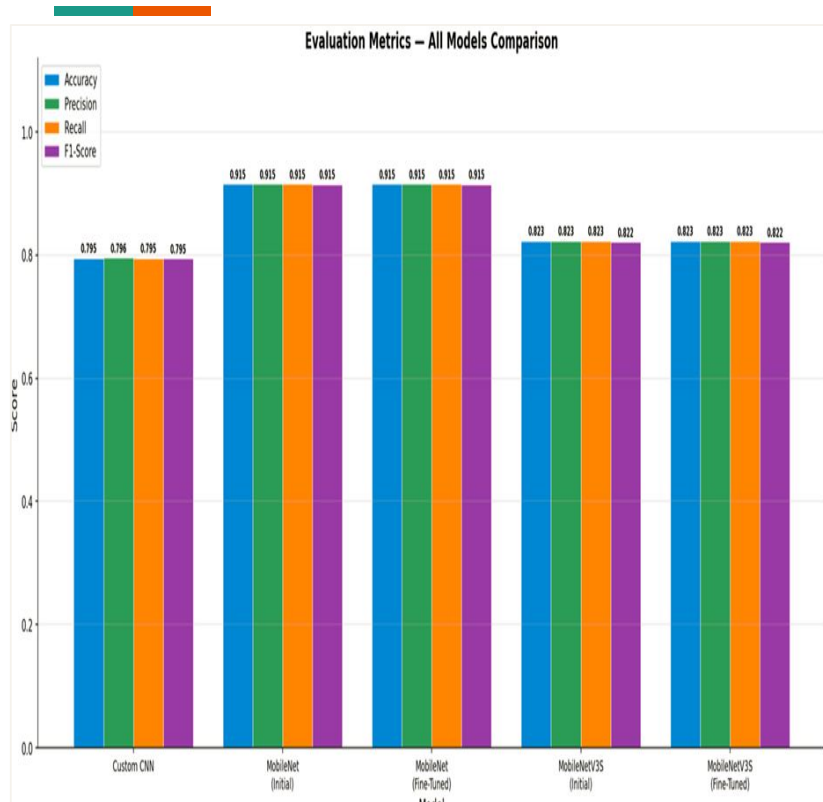


Model	Type
Custom CNN	Built from scratch
MobileNet (Initial)	Transfer learning, frozen base
MobileNet (Fine-Tuned)	Transfer learning, last 30 layers unfrozen
MobileNetV3Small (Initial)	Transfer learning, frozen base
MobileNetV3Small (Fine-Tuned)	Transfer learning, last 20 layers unfrozen

Technical challenges

- **Image resolution mismatch** — CIFAR-10 is 32×32 , MobileNet needs 96×96 . Solved with `tf.image.resize` inside the pipeline
- **RAM out-of-memory crash** — Resizing 50,000 images multiplied RAM $9\times$. Solved with lazy `tf.data` pipelines (only 1 batch in RAM at a time)
- **Mixed precision training** — `float16` policy cut memory $\sim 50\%$ with no accuracy loss
- **Reduced batch size** — from 32 $\rightarrow$ 16 to further reduce peak memory

Evaluation Results



Model	Accuracy	Precision	Recall	F1-Score
Custom CNN	0.7952	0.7961	0.7952	0.7948
MobileNet (Initial)	0.9155	0.9153	0.9155	0.9151
MobileNet (Fine-Tuned)	0.9155	0.9153	0.9155	0.9151
MobileNetV3Small (Initial)	0.8233	0.8231	0.8233	0.8216
MobileNetV3Small (Fine-Tuned)	0.8233	0.8231	0.8233	0.8216

Key Insights to Discuss

- **cat vs dog** and **automobile vs truck** are typically the hardest classes — your confusion matrix will show this
- Fine-tuning always improves over frozen transfer learning — show the delta
- Transfer learning outperforms a custom CNN trained from scratch, even on a small dataset like CIFAR-10
- The `tf.data` pipeline approach is production-standard and scales to any dataset size



Summary

Comparison on testing data:
Custom CNN: Limited capacity on
feature extraction (high losses)

Model	F1-score
Custom-CNN	0.73
EfficientNetB0	0.88
MobileNetV3	0.92
DenseNet121	0.95